Neural Networks and Deep Learning:
Part 1: Theoretical Foundations

DIGIT RECOGNITION

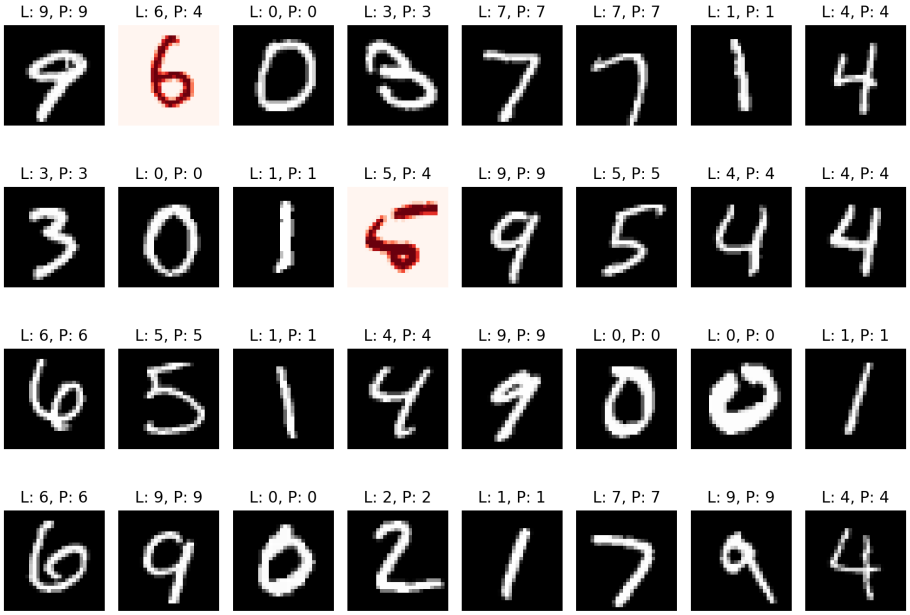
Model comparison and performance evaluation for an MLP

Giovanni Matteazzi

g.matteazzi@santannapisa.it

Scuola Superiore Sant'Anna, Pisa

April 2026



Contents

1 Introduction **3**

2 Training Regimes **3**
2.1 Early stopping . . . 3
2.2 Accuracy and Loss . . . 4
2.3 Learning rate vs Batch size . . . 4
2.4 Batch modes . . . 5

3 Optimizer **6**
3.1 Learning rate scheduler . . . 6
3.2 Momentum . . . 7

4 Loss function **8**
4.1 L1, L2 . . . 8
4.2 Regularization . . . 9

5 Architectural Components **10**
5.1 Dropout and Batch Normalization . . . 10
5.2 Weights initialization . . . 10
5.3 Activation function . . . 12

6 Data Augmentation **13**

7 Summary **14**

8 Best model evaluation **14**
8.1 All metrics . . . 14
8.2 Playground interface . . . 16

9 Conclusion **16**

1 Introduction

This work investigates the use of a Multilayer Perceptron (MLP) for handwritten **digit recognition** on the MNIST dataset, which consists of 60,000 grayscale samples.

The goal is to systematically compare different hyperparameter configurations and training techniques through quantitative analysis and visualization. The best-performing models are then evaluated using standard metrics and applied to a simple interactive interface.

All experiments are implemented in Python using the PyTorch framework.

Since the hyperparameter space is huge, the validation of the study might be compromised by comparing unsuitable configurations. Therefore, variables are modified in coherent groups while carefully considering their interdependencies. In addition, experiments are conducted in a deterministic setting to ensure reproducibility and consistency of results.

2 Training Regimes

To isolate the effect of individual factors, the model architecture is kept fixed throughout the whole simulation. The network is a fully connected MLP with two hidden layers of 128 and 64 neurons.

Given that MNIST images are 28×28 grayscale pixels, the input layer consists of 784 units, while the output layer has 10 neurons corresponding to the digit classes.

This configuration provides a good trade-off between model capacity and generalization: it uses approximately 110k parameters (weights and biases), whereas, for example, a single hidden layer with 256 neurons would nearly double the parameter count (∼202k).

The baseline configuration, then, uses ReLU activations, Stochastic Gradient Descent (SGD) as optimizer, and Cross-Entropy Loss, which are standard choices for classification tasks.

2.1 Early stopping

To ensure fair comparisons across configurations, **early stopping** is employed in the majority of tests. This technique stops training when the validation accuracy is not improving anymore - while training accuracy continues to improve, indicating overfitting.

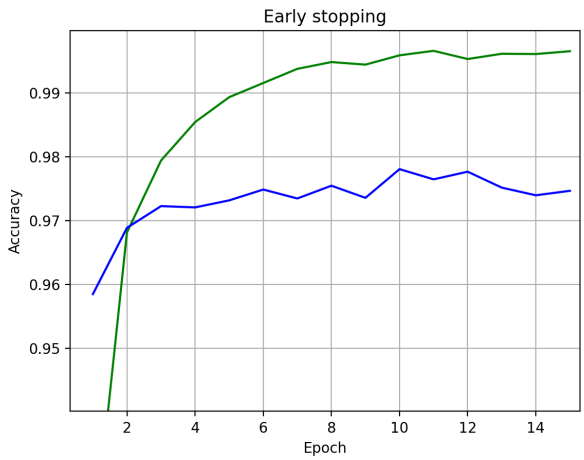


Figure 1: Accuracy from training (green) and validation (blue). In this case, the network will save model's parameters reached in epoch 10, since in last 5 periods validation accuracy is stationary (while training one keeps growing).

2.2 Accuracy and Loss

The primary evaluation metrics are **accuracy** and **loss**. Since MNIST is a balanced dataset, precision and recall provide limited additional insight in this context.

Accuracy is defined as the proportion of correctly classified samples, while loss clearly depends on the chosen objective function.

A key hyperparameter in supervised learning is **batch size**, which determines how the dataset is partitioned during training, validation and test. This gives us the opportunity to compare the behavior of loss and accuracy with very different training approaches, based, indeed, on very different batch sizes.

However, batch size is strongly coupled with **learning rate**, and their interaction must be carefully considered preliminarily.

2.3 Learning rate vs Batch size

To identify suitable learning rates for each batch size, a **grid search** is performed on a set of candidate values for BS and LR.

To reduce computational cost, early stopping is configured with max_epochs = 20 and patience = 2. In later experiments, these values are increased to improve convergence quality.

Note: Due to computational constraints, results are based on single runs rather than averages over multiple tests. For this reason, determinism is considered crucial throughout the report.

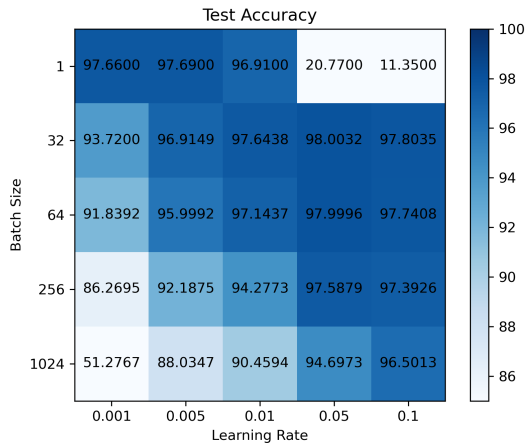


Figure 2: Heatmap with BS $\in \{1, 32, 64, 256, 1024\}$, LR $\in \{0.001, 0.005, 0.01, 0.05, 0.1\}$.

2.4 Batch modes

At this point, we want to compare three training regimes:

- Online (BS $= 1 = 2^0$)
- Mini-batch (BS $= 32 = 2^5$)
- Batch (BS $= 1024 = 2^{10}$)

Each configuration is paired with its most suitable learning rate (0.005, 0.05, and 0.1 respectively).

Note: Smaller batch sizes require smaller learning rates, as updates are more frequent and noisier.

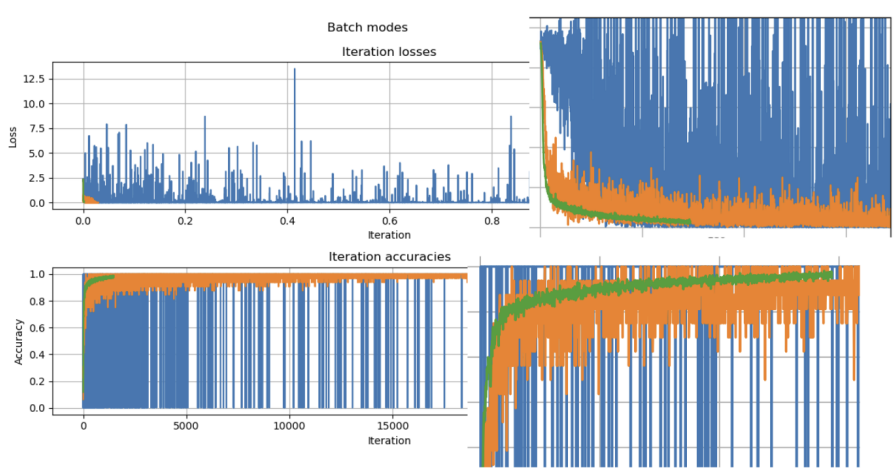


Figure 3: Loss and accuracy over iterations for different batch modes (batch in green, mini-batch in orange, online in blue).

Despite the plot appearing confusing at first, if we zoom on the first iterations we can distinctly notice the smoother behavior of batch mode and the bigger variance, until end of training, of online learning. However, as final test accuracy is regarded, online learning achieves slightly higher final accuracy (98.10%) compared to mini-batch (98.05%) and far better than batch (97.17%).

For computational efficiency, subsequent experiments use BS = 32.

3 Optimizer

With the batch size fixed, experiments are conducted using a consistent dataset partitioning across runs.

The only source of randomness is the data loader, which shuffles samples at each epoch (although always in the same way on every test). This prevents bias due to sample ordering.

Determinism is verified by repeating identical experiments and observing consistent results.

```
CONFIG: lr = 0.05, bs = 32, act_fun = ReLU, opt = SGD, loss_fun = CrossEntropyLoss
TIMESTAMP: 1775921527.7685895
EPOCH 1:
Training: loss = 0.326696987393159, accuracy = 0.9032509596928983
Validation: loss = 0.16203187946111156, accuracy = 0.952176517571885
EPOCH 2:
Training: loss = 0.12535827889798956, accuracy = 0.9615523032629558
Validation: loss = 0.11045319717615462, accuracy = 0.9665535143769968
```

```
CONFIG: lr = 0.05, bs = 32, act_fun = ReLU, opt = SGD, loss_fun = CrossEntropyLoss
TIMESTAMP: 1775921590.2071042
EPOCH 1:
Training: loss = 0.326696987393159, accuracy = 0.9032509596928983
Validation: loss = 0.16203187946111156, accuracy = 0.952176517571885
EPOCH 2:
Training: loss = 0.12535827889798956, accuracy = 0.9615523032629558
Validation: loss = 0.11045319717615462, accuracy = 0.9665535143769968
```

Figure 4: Same configurations give the same result (note the different timestamps).

3.1 Learning rate scheduler

We have already discussed learning rate as a variable and the way it affects the convergence of the network. A learning rate **scheduler** is now introduced to improve convergence.

The intuition is that higher learning rates are beneficial in early training stages, while smaller values are required near convergence for stability and precision.

The scheduler applies exponential decay: $\eta_i = \eta_{i-1} \cdot \gamma$ every k epochs.

In our setup $\eta_0 = 0.1$, $\gamma = 0.5$ and $k = 5$, which means that LR is 0.1 on epochs 1 to 5, then decreases to 0.05 in 6-10, then 0.025 in 11-15 and so on.

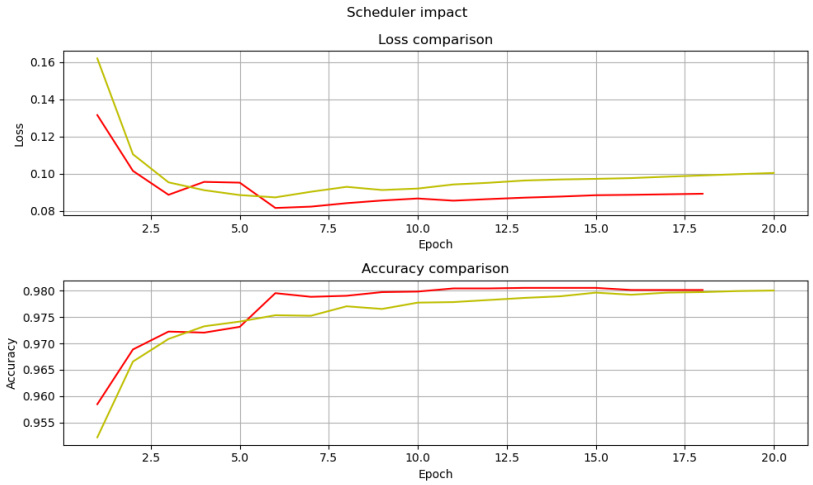


Figure 5: Loss and accuracy over epochs with scheduler (red) and without (yellow).

By introducing a learning rate scheduler in the baseline configuration, the model achieves a test accuracy of 98.13%. Notice that the best-performing model is reached as early as epoch 15, whereas the baseline without scheduling requires up to epoch 27.

The training curves also exhibit less smooth behavior around epoch 5, corresponding to the first decay step in the learning rate. This reflects the sudden change in optimization dynamics induced by the scheduler.

The results suggest that the scheduler accelerates convergence, although its parameters could be further fine-tuned to achieve smoother transitions and potentially improve performance.

3.2 Momentum

We now investigate another feature offered by SGD optimizer: **momentum**.

Momentum introduces to weights update a term proportional to the previous one, effectively smoothing the optimization trajectory and helping escape shallow local minima.

The update rule is:

$\Delta w_t = \mu \Delta w_{t-1} - \eta \Delta E(w_{t-1})$

A typical value for momentum μ is 0.9, testing its accuracy across different scale of learning rates we obtain the results on the right.

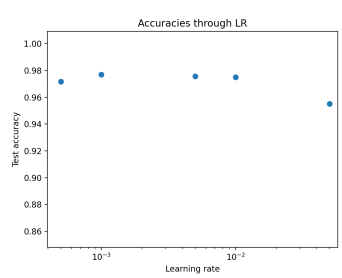


Figure 6: Test accuracy with fixed momentum $\mu = 0.9$ and varying learning rate $\in \{0.0005, 0.001, 0.005, 0.01, 0.05\}$.

Despite testing momentum with the most suitable LR found, 0.001, it does not improve performance and slightly reduces accuracy (97.80%). This may be due to the relatively simple loss landscape and the additional excessive instability introduced.

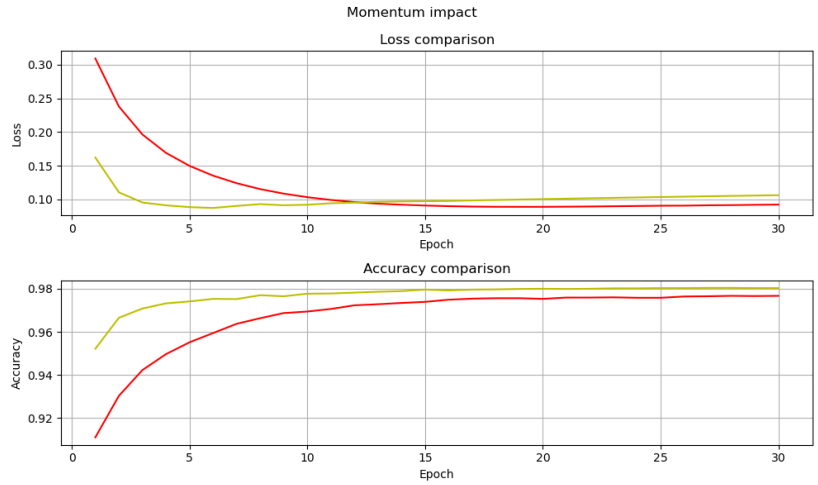


Figure 7: Validation losses and accuracies comparison (in red with momentum, basic network in yellow).

4 Loss function

The baseline loss function we have used so far is **Cross-Entropy Loss**, which is standard for multi-class classification tasks.

It is defined as: $E = -\sum_{i=1}^{C} t_i log(y_i)$ where t is the one-hot encoded target vector and y is the softmax output.

4.1 L1, L2

We evaluate **L1-norm loss** (Mean Absolute Error) and **L2-norm loss** (Mean Squared Error) in this classification setting. The results show that both are not well suited for the task, achieving test accuracies of 96.61% and 97.22%, respectively.

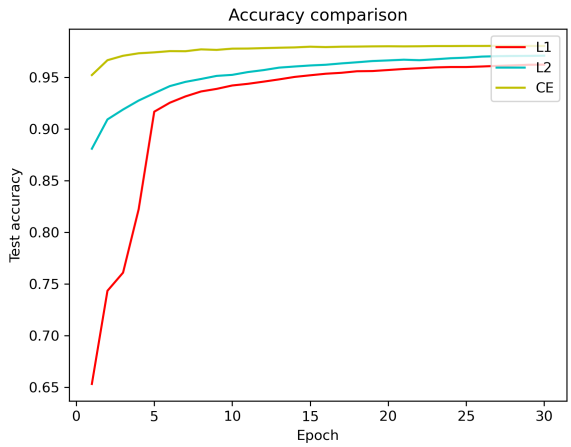


Figure 8: Accuracy through epochs for different loss functions; loss is clearly not comparable. For L1 and L2, learning rate is equal to 0.1.

4.2 Regularization

Still, we can try out their regularized version. **Regularization** aims to reduce **overfitting** by penalizing large weights, thereby improving generalization performance.

In particular, L1 regularization adds a term of the form $\frac{\gamma}{m}\sum|w|$, while L2 regularization adds $\frac{\gamma}{2m}\sum w^2$, where m is the batch size, w denotes the weight vector, and γ is the regularization coefficient.

This encourages the network to simultaneously minimize the loss and constrain the magnitude of the weights, promoting more stable and generalizable solutions.

These are the results with regularization applied.

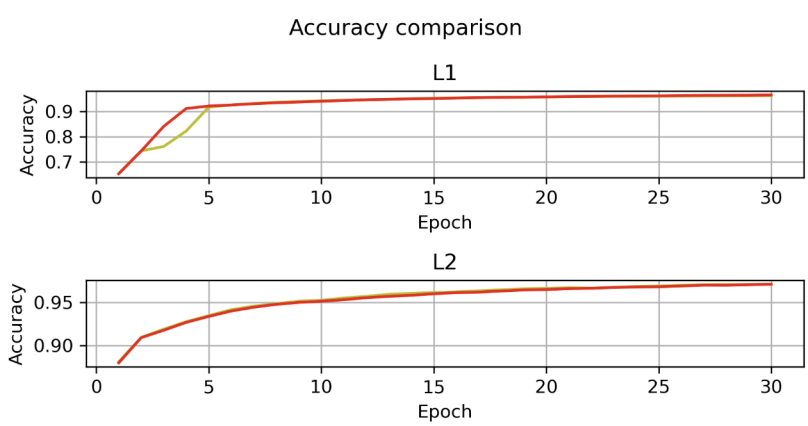


Figure 9: Regularization impact.

After limited fine-tuning (further changes in γ or the learning rate lead to worse

performance), the effect is observable but marginal. In particular, test accuracy improves by less than 0.10%: regularization techniques show limited impact due to the simplicity of the dataset.

5 Architectural Components

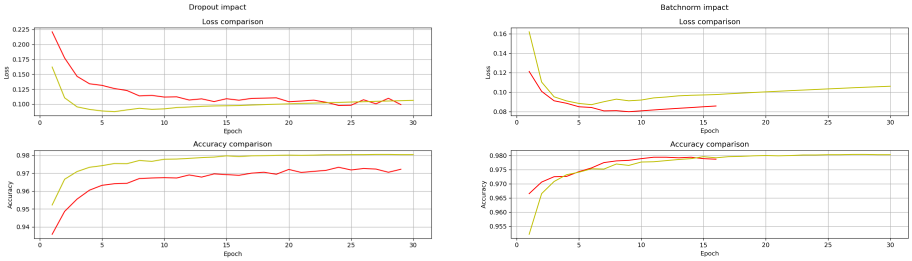
Keeping the architecture fixed, we now analyze additional components using a **One-Factor-At-a-Time** (OFAT) approach.

5.1 Dropout and Batch Normalization

Speaking of regularization, two common techniques can be introduced to improve generalization and training stability: dropout and batch normalization.

Dropout randomly deactivates a subset of neurons during training, reducing co-adaptation between units and preventing overfitting. However, given the relatively small model and dataset, its effect is limited and slightly degrades performance, resulting in a test accuracy of 97.36%.

Batch normalization, on the other hand, stabilizes training by normalizing intermediate activations, improving gradient flow and granting a quicker convergence. As observed in the experiments, indeed, the model reaches peak performance significantly faster (13 epochs compared to 27), with a test accuracy of 97.88%.



(a) Dropout impact. **(b)** Batch normalization impact.

Figure 10

5.2 Weights initialization

So far, weights have been initialized using the default **Kaiming** initialization, which is well suited for ReLU-based architectures due to its variance scaling properties.

A naive alternative is to initialize all weights to a constant value, such as **zero**. However, this prevents learning entirely: all neurons remain symmetric and receive identical gradients, effectively behave as a single unit. As a result, the model fails to learn, with accuracy remaining around 10%, equivalent to randomly guessing the digit.

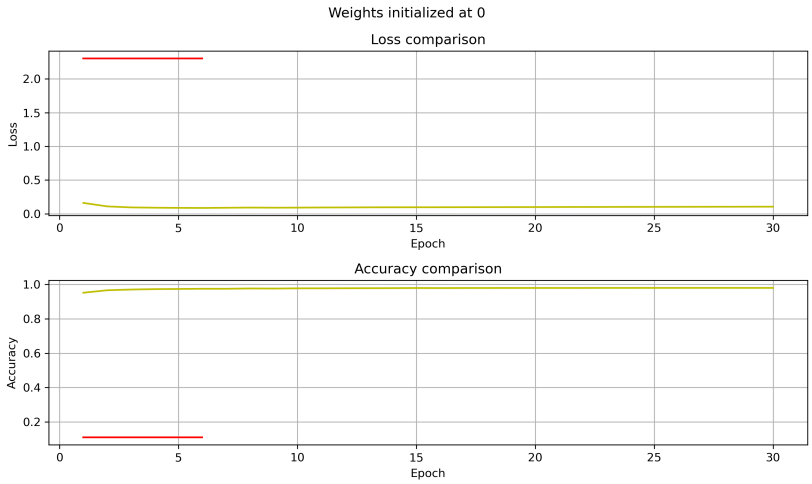


Figure 11: Weights are initialized all equal to zero.

A more reasonable alternative is to sample weights from a **normal** distribution with fixed variance (*e.g.* 0.01). Compared to Kaiming initialization (which would have a higher variance of 0.05 with this setting), this approach performs slightly worse, achieving a test accuracy of 97.95%.

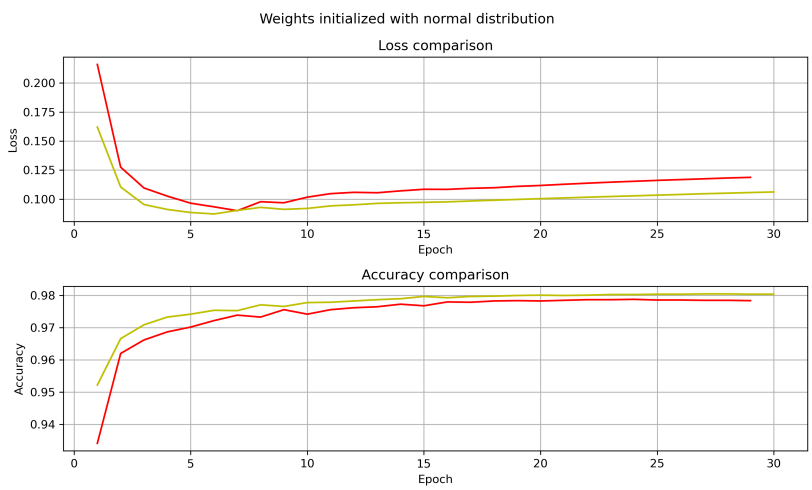


Figure 12: Losses and accuracy for the network with normal initialized weights.

Finally, we evaluate **Xavier** initialization, which scales variance based on the number of input and output units. This method achieves a test accuracy of 97.73%.

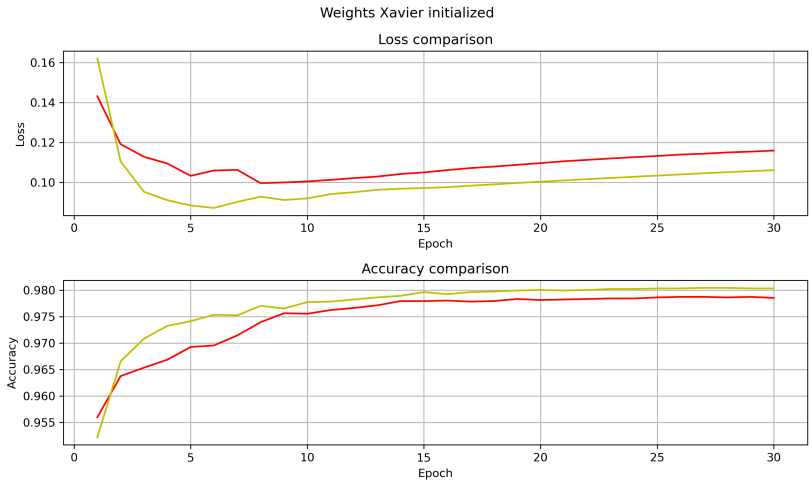


Figure 13: Weights have Xavier initialization.

5.3 Activation function

The baseline model uses the **ReLU** activation function, which mitigates the vanishing gradient problem by maintaining a constant gradient for positive inputs.

A similar variant, **LeakyReLU**, introduces a small slope for negative inputs (typically 0.01), preventing zero gradients and reducing the risk of dead neurons. This leads to a slight improvement, achieving a test accuracy of 98.10%.

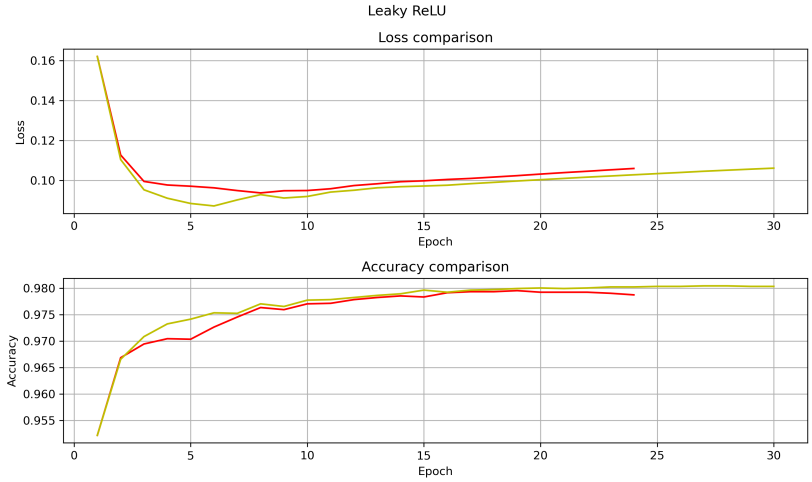
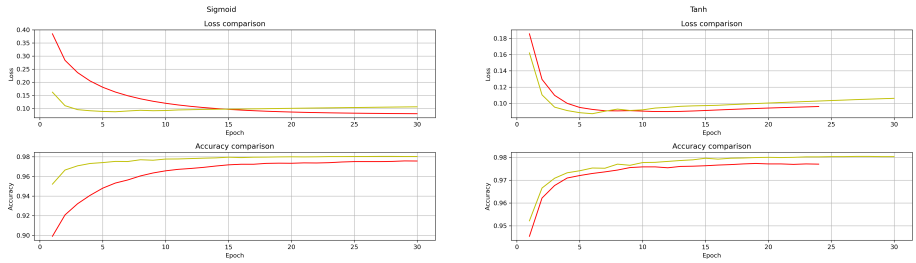


Figure 14: Activation function is now LeakyReLU.

We also evaluate **sigmoid** and **tanh** activation functions. Due to their symmetry, they are paired with Xavier initialization. However, results confirm that ReLU-based

activations are more effective in this task.



(a) Sigmoid impact. **(b)** Tanh impact.

Figure 15

6 Data Augmentation

The final method investigated shifts the focus from the model architecture to the training procedure itself. **Data augmentation** refers to a set of techniques aimed at improving generalization by introducing controlled variability into training data.

The following random transformations are applied:

- rotation of a maximum of $\pm 15°$
- translation up to 5%
- scaling of ± 0.10 maximum
- skewing up to 5%

Additionally, random noise is introduced as: $x = x + 0.15 \cdot rand()$.

With these light transformations, the model achieves an accuracy of 98.80% in the first test, which slightly decreases to 98.61% when noise is added.

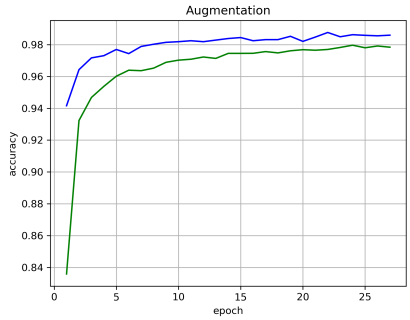


Figure 16: Training (green) and validation (blue) accuracy over epochs for the augmented dataset without noise.

It is worth noting that validation accuracy remains consistently higher than training accuracy: this is expected, as validation samples are not augmented and therefore easier for the network. More importantly, the absence of divergence between training and validation curves suggests that overfitting is effectively mitigated.

Apparently, data augmentation leads to a clear improvement in generalization performance, increasing the robustness of the network to geometric variations and small perturbations in the input data.

7 Summary

We evaluated multiple hyperparameter configurations and training techniques for the digit recognition task, some more successful than others. Based on the results, we can compare their relative impact on performance.

Here are the **test accuracies** gathered:

Test	**Accuracy (%)**
BS = 1, LR = 0.005	98.10
BS = 32, LR = 0.05	98.05
BS = 1024, LR = 0.1	97.17
scheduler	98.13
momentum	97.80
L1	96.61
L1 with regularization	96.70
L2	97.22
L2 with regularization	97.27
dropout	97.36
batch normalization	97.88
zero initialization	11.35
normal initialization	97.95
xavier initialization	97.73
LeakyReLU	98.10
sigmoid	97.70
tanh	97.65
data augmentation	98.80
data augmentation with noise	98.61

Table 1: Test accuracies obtained under consistent training conditions (early stopping with max_epochs = 30 and patience = 5). Colors vary from dark red (very bad) to light green (very good).

It's easy to realize that **learning rate scheduling** and the use of **LeakyReLU** provide good improvements, whereas **data augmentation** produces by far the most significant gain in terms of generalization and final accuracy.

8 Best model evaluation

8.1 All metrics

Treating data augmentation as an additional training-time regularization strategy, rather than as part of the core model configuration, we evaluate the best-performing setup: the baseline with BS = 32 and LR = 0.05, enhanced with a learning rate scheduler and LeakyReLU activation. Let us examine all his metrics:

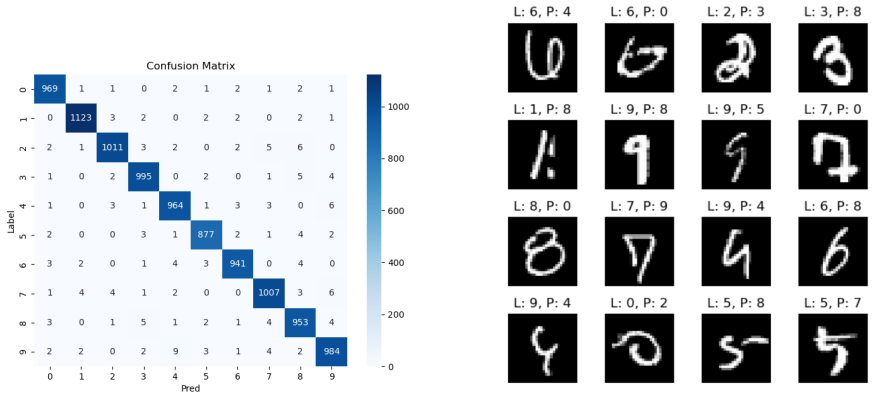
```
Classification Report:

              precision    recall  f1-score   support

           0     0.9848    0.9888    0.9868       980
           1     0.9912    0.9894    0.9903      1135
           2     0.9863    0.9797    0.9830      1032
           3     0.9822    0.9851    0.9837      1010
           4     0.9787    0.9817    0.9802       982
           5     0.9843    0.9832    0.9837       892
           6     0.9864    0.9823    0.9843       958
           7     0.9815    0.9796    0.9805      1028
           8     0.9715    0.9784    0.9749       974
           9     0.9762    0.9752    0.9757      1009

    accuracy                         0.9824     10000
   macro avg     0.9823    0.9823    0.9823     10000
weighted avg     0.9824    0.9824    0.9824     10000

Precision: 0.9823
Recall: 0.9823
F1 Score: 0.9823


ACCURACY: 0.9824
```

Figure 17: Classification report for the described network.

The model achieves a test accuracy of **98.24%**. As expected, precision and recall are not that informative due to the balanced nature of the dataset (look at the "support" column).

As an interesting statistic, from the classification report we see that the most frequently **misclassified digits** are 4, 7, 8, and 9. The confusion matrix (Fig. 18a) highlights common confusions, particularly between pairs such as 4–9, 3–8, and 7–9.

Examples of misclassified samples (Fig. 18b) show that some of them are ambiguous and borderline even for human interpretation.



(a) **(b)**

Figure 18

8.2 Playground interface

The model can also be interactively tested through a simple **interface**, where users can draw digits and observe predicted probabilities.

Figure 19: Example of correct prediction with proper confidence value for each digit.

It is worth noting that the model appears less accurate with user input, with occasional misclassifications even for relatively clear digits.

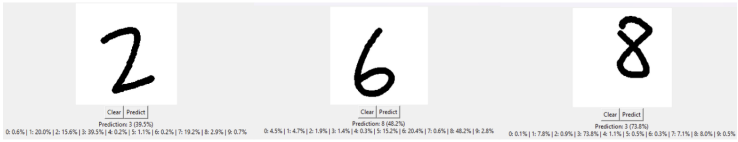


Figure 20: Example of incorrect predictions. Also compare confidences, got after a softmax operation.

9 Conclusion

With the available tools, the experimental results are overall satisfactory and, when properly interpreted, clearly highlight the connection between theoretical concepts and practical implementation.

Although the task is relatively simple for a human — who would achieve near-perfect accuracy — the fact that a model can learn to recognize handwritten digits with a high level of confidence, even from user-generated input, is far from being trivial.

Furthermore, the result obtained on MNIST - with the highest being a test accuracy of 98.80% - is impressive.

However, it remains slightly inferior to what can be achieved with larger architectures or convolutional models such as LeNet-5, that even reached 98.95% (see all models in https://github.com/giovamatteazzi/Digit-recognition). This naturally suggests that exploring deeper architectures and convolutional approaches could represent a promising direction for future work.

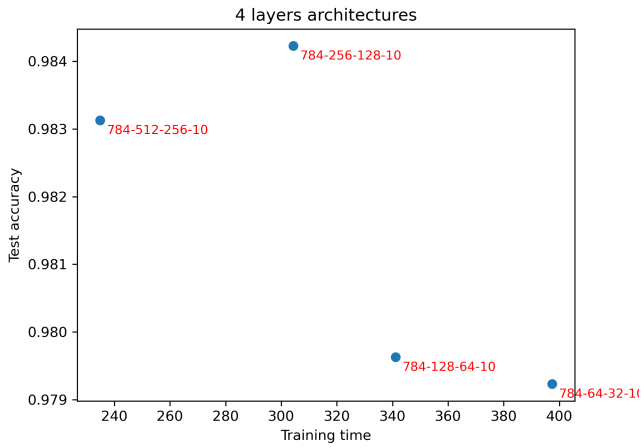


Figure 21: Test accuracy over training time for the considered models.

As a first step, we analyze how the previously identified best-performing model behaves under slight architectural variations. Specifically, we evaluate the following configurations of four layers each:

- 784 - 64 - 32 - 10
- 784 - 128 - 64 - 10
- 784 - 256 - 128 - 10
- 784 - 512 - 256 - 10

comparing both final accuracy and, for the first time, training time. Another important research direction concerns training and inference optimization—note that all experiments in this work were conducted on a standard CPU. Here, we limit ourselves to investigating whether increasing the number of trainable parameters leads to better performance.

As we can see, increasing model size does affect performance, but not in a straightforward way. Paradoxically, larger architectures require less time, as they converge faster (despite longer epochs), yet additional neurons do not always translate into proportional accuracy gains: architecture must be carefully tuned and adapted to the specific task and constraints, requiring a balance between model complexity, training efficiency, and generalization capability.